COMPSCI 2XC3 Final Lab Report

Prepared by

Group 64

Luca Mawyin

Anderson Ray

Theodore Pham

COMPSCI 2ME3

McMaster University

April 4, 2026

Contents

Part 1 . . . 3
- Experiment Suite 1 . . . 3
  - Experiment 1 . . . 3
  - Experiment 2 . . . 4
  - Experiment 3 . . . 6
- Mystery Algorithm . . . 7
  - All Shortest Paths Algorithm . . . 7
  - Graph Complexity Analysis . . . 7
  - Mystery Algorithm Analysis . . . 7

Part 2 . . . 8
- Discussion . . . 8
  - How A* works . . . 8
  - Issues A* addresses . . . 8
  - Empirically testing A* against Dijkstra's algorithm . . . 8
  - Effects of an arbitrary heuristic function . . . 9
  - Applications where A* should be used instead of Dijkstra's algorithm . . . 9

Part 3 . . . 9
- Data and Graph Construction . . . 9
- Heuristic Function . . . 9
- Experimental Setup . . . 9
- Results . . . 10
- Discussion . . . 10
- Limitations . . . 11
- Conclusion . . . 11

Part 4 . . . 12
- Discussion . . . 12
  - Figure 2's design principles and patterns . . . 12
  - Robustness of the future implementation's of different node representation . . . 13
  - Potential Graph's that can be implemented . . . 13

Appendix . . . 14
- Additional Figures for Part 3 . . . 14
- Code Navigation . . . 15

List of Figures

1 Distance Error of Approximation Algorithms with Increasing k 3
2 Distance Error of Approximation Algorithms with Increasing Density 5
3 Distance Error of Approximation Algorithms with Increasing Graph Size 6
4 Log-Log Runtime Comparison of Varying Algorithms 8
5 P3_Median runtime by transfer category. A* is faster in all three groups. 11
6 P3_Median runtime by path length. A* remains faster across almost all path lengths. 12
7 P3_A* speedup by transfer category. 12
8 P3_Downsampled runtime comparison of Dijkstra and A*. 14
9 P3_Median runtime by exact number of transfers. 14

Part 1

Experiment Suite 1

Experiment 1

Objective

In this experiment, we will compare both `dijkstra_approx()` and `bellman_ford_approx()` to `dijkstra()` and `bellman_ford()` respectively, with increasing values of k. This experiment was chosen in order to test if the approximation algorithms converge to their true counterparts when increasing k of the approximation algorithms, and at what rate these changes occur at. The conditions for the experiment are outlined as follows:

- Generate a sparse graph of 1000 nodes, with edge weights between 1 and 100
- Each edge should be generated with a probability of 5%
- Values of k will increase from 1 to 20, with a step size of 1
- For each value of k, we will calculate the total distance error between the approximation algorithms and the true algorithms.

Results

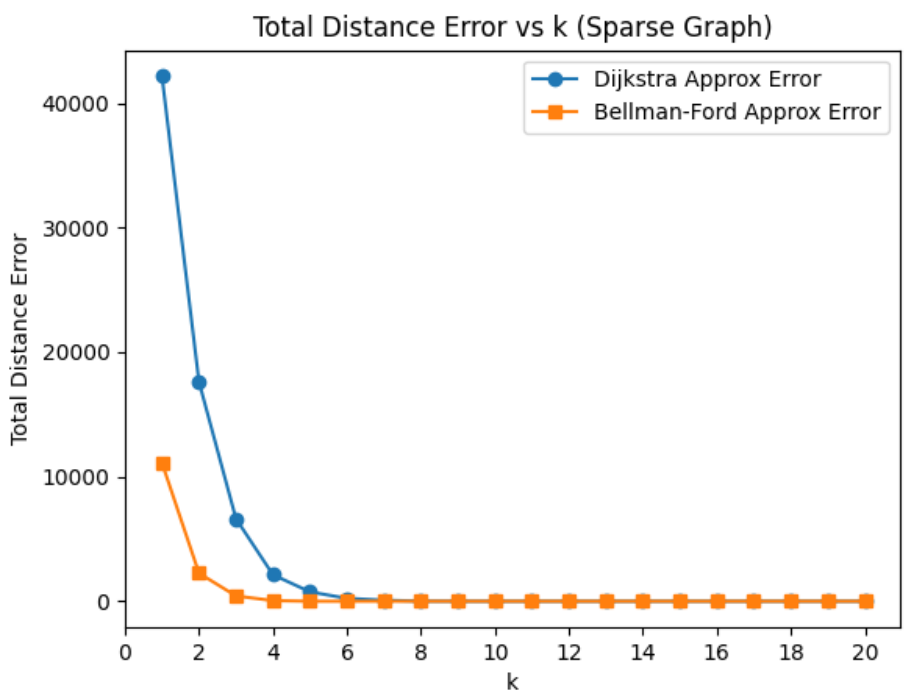

Figure 1: Distance Error of Approximation Algorithms with Increasing k

Discussion

Observing the graph, we can see that as k increases, the average node distance error of both approximation algorithms decreases. However, this approximation error swiftly decreases, and plateaus by $k = 8$ for Dijkstra, and $k = 5$ for Bellman-Ford.

Considering the Dijkstra's approximation algorithm, this plateau is likely due to the fact that the approximation algorithm functions by performing Dijkstra's algorithm on the subgraph of the k nearest neighbours of the current node being iterated through. This means that as k increases, the approximation algorithm considers a much larger portion of the graph, therefore resulting in a much closer approximation to the true Dijkstra's algorithm.

Observing the Bellman-Ford approximation algorithm, this plateau is likely due to the nature of how the algorithm functions. The Bellman-Ford approximation algorithm functions by searching for the shortest path from the source node to the target using only k edges, with k increasing iteratively. Therefore on a sparse graph, as there are not many edges, the shortest path from the source to the target node is likely to be found within a very small number of edges, resulting in a very early plateau in the approximation error.

Experiment 2

Objective

In this experiment, we will compare both `dijkstra_approx()` and `bellman_ford_approx()` to `dijkstra()` and `bellman_ford()` respectively, with increasing graph density, and fixed values of k. The conditions for the experiment are outlined as follows:

- Graph density will increase from 1% to 10%, with a step size of 1% each iteration
- For each graph density, generate a sparse graph with 500 nodes, with edge weights between 1 and 10
- Values of k will be 1, 3 and 5
- For each graph density and value of k, we will calculate the total distance error between the approximate and true algorithms

Results

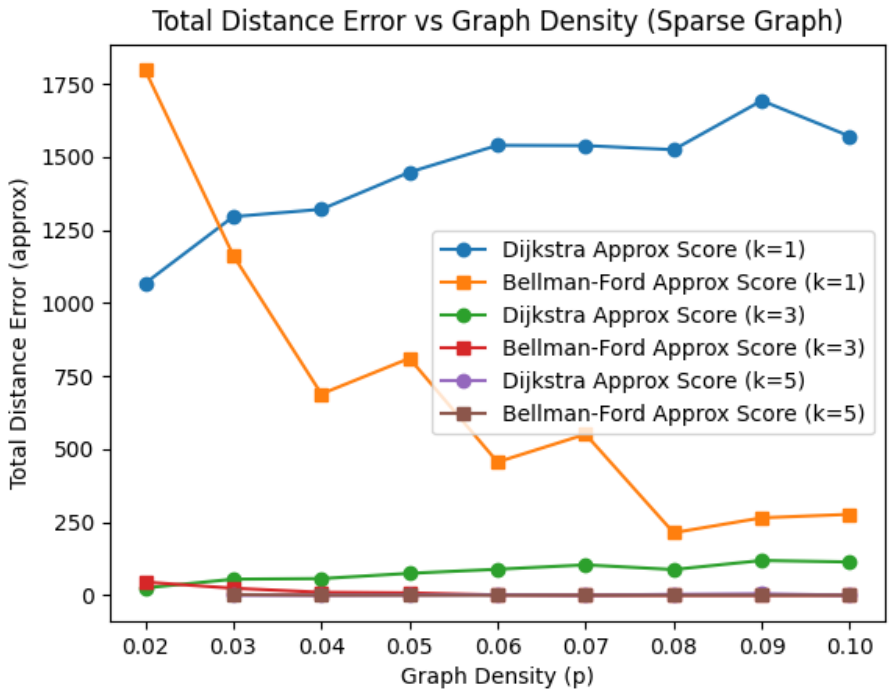

Figure 2: Distance Error of Approximation Algorithms with Increasing Density

Discussion

Observing the graph, we can see that as graph density increases for all values of k, the distance error for the Bellman-Ford approximation decreases, while the distance error for Dijkstra's approximation increases.

The reason for which the distance error for the Bellman-Ford approximation algorithm decreases is likely due to the fact that, as there are more edges present in the graph, there is more likely to be a more direct connection between two nodes. This increase in density within the graph means that the shortest path between nodes becomes shorter.

The reason for which the distance error for Dijkstra's approximation algorithm increases is likely due to how the approximation algorithm functions. The approximation function for Dijkstra's is greedy. When a graph is less dense, each node has less edges, meaning that there are less candidate paths to explore, which will allow us to stay below our k value. However, when a graph is much more dense, there are many more possible paths to check, meaning that we are likely to reach our threshold of relaxing earlier. This prevents us from finding more efficient paths, as a node cannot be updated once it reaches the threshold of k.

Ultimately, when we are limiting k for each algorithm, the Bellman-Ford algorithm seems to perform better in more dense graphs, as the Total Distance Error is substantially lower than that of the Dijkstra approximation.

Experiment 3

Objective

In this experiment, we will compare both `dijkstra_approx()` and `bellman_ford_approx()` to `dijkstra()` and `bellman_ford()` respectively, with increasing graph sizes, and fixed values of k. This experiment aims to highlight the trends of relative distance error between the approximation algorithms and their true counterparts, and observe how the number of nodes affects the algorithms ability to approximate the shortest paths. The conditions for the experiment are outlined as follows:

- Graph size will increase from 100 nodes to 500 nodes, incrementing by 50 each iteration
- The density will remain constant, at 5%, with edge weights between 1 and 10
- Values of k will be 1, 3, and 5
- For each difference between the true and approximate error, we will normalize it with the number of nodes in the graph

Results

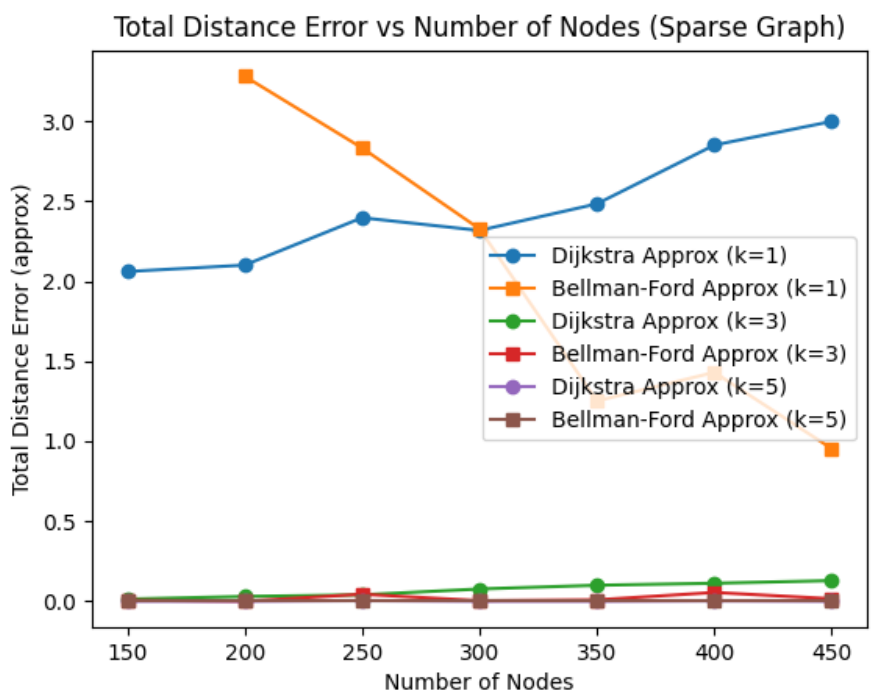

Figure 3: Distance Error of Approximation Algorithms with Increasing Graph Size

Discussion

Observing the graph, it is evident that as the number of nodes increases, the relative distance error of the Bellman-Ford approximation algorithm decreases, while that of the Dijkstra approximation increases.

The reason for which the relative distance error of the Bellman-Ford approximation algorithm decreases is likely for the same reason that it decreases when increasing graph density. As the graph density remains at 5%, when the number of nodes increase, the number of edges also greatly increase. This means that larger graphs become very well connected, and as a result, shortest paths between nodes do not become much

larger. This means that nodes are still reachable in few steps, resulting in the graph size not hindering the Bellman-Ford approximation algorithm.

The increasing number of edges for the graph is also why the relative distance error for Dijkstra's approximation increases on larger graphs. When graphs are smaller, Dijkstra's approximation algorithm is able to exhaust a larger relative portion of the graph before reaching the limits given for the function. However on larger graphs, as there are more nodes and edges, it will reach these limits before it is able to explore a substantial relative portion of the graph. This means that only a small portion of the distance in a route can be relaxed, which becomes less substantial when the graph, and subsequently paths, are longer.

Mystery Algorithm

All Shortest Paths Algorithm

For designing an all-pairs-shortest path algorithm for a graph with positive edge weights, we can use Dijkstra's algorithm, iteratively. We can iterate through each node in our graph, with each source being the current node being iterated. In a similar logic, when introducing negative weights into the graph, we can use Bellman-Ford's algorithm iteratively for each node in the graph.

Graph Complexity Analysis

Given that Dijkstra has complexity $\Theta(E + V \log V)$, or $\Theta(V^2)$ if the graph is dense, for the all-pairs-shortest path algorithm using Dijkstra the time complexity would be $\Theta(V^3)$. This is because as we need to iterate through each node in the graph, we also perform Dijkstra V times. Similarly to Dijkstra, the all-pairs-shortest path algorithm implementing Bellman-Ford would have a time complexity of $\Theta(V^4)$. As stated before, as we iterate through each node in the graph, we must perform Bellman-Ford V times, subsequently increasing our runtime to $\Theta(V^4)$.

Mystery Algorithm Analysis

We are given a mystery algority `mystery(G)`, and are expected to analyze how it functions. Observing the function, it is evident that it calculates the shortest distance between every pair of nodes in the graph. This is done by creating a 2D matrix of nodes where d[i][j] represents the shortest distance from i to j. For every node i, for every node j, the algorithm iterates through node k to determine if it results in a shorter path from i to j, and updates the matrix accordingly. To add some context for runtime, for every node in the graph, we are iterating through every other node in the graph. With this pair of nodes, we are now iterating through every remaining node to see if the distance from the first node to the second, and the second node to the third, is a shorter distance than from the first node to the second. This implies that the upper bound of the algorithm is likely $\Theta(V^3)$. This also becomes more evident when running a Log-Log Runtime Comparison using dense graphs as shown in Figure 4, as the runtime appears to scale similiarly to that of Dijkstra.

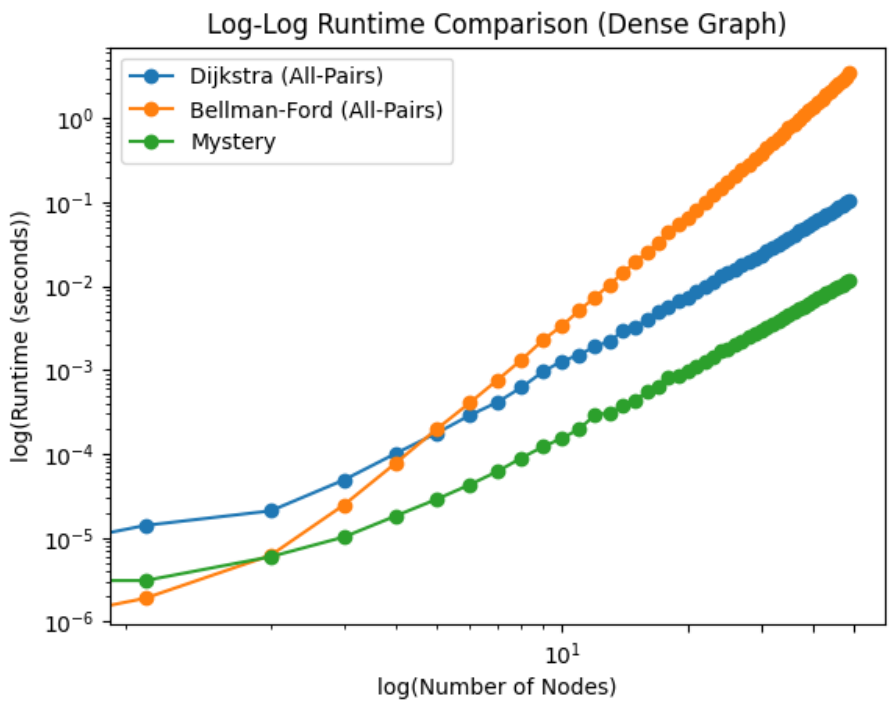

Figure 4: Log-Log Runtime Comparison of Varying Algorithms

Part 2

Discussion

How A* works

The A* Algorithm is an algorithm used find the shortest path from a start node to a target node in a directed weighted graph. Its algorithm is practically the same as Dijkstra's, the only difference is how it determines priority (the key value) in the priority queue.

In A*, the key value used in the priority queue is equal to the total weight along the path traversed (like Dijkstra's), plus some value coming from a heuristic function. This heurisitc function outputs a number relative to its distance from the destination, guiding the algorithm to explore nodes that are closer to the destination and more likley to be along the shortest path.

Issues A* addresses

A* addresses a problem that Dijkstra has where it explores nodes that are clearly not on the shortest path. For example, if we were to calculate the shortest route from Toronto to Hamilton with Dijktra's it'll probably venture out to Scarborough before finding Hamilton, which is obviously inefficient. If we were to use A*, we can use a heurisitc function that calculates the straight line distance to Hamilton. This heurisitc function would make A* prioritize exploring western locations relative to Toronto, aside from exploring Scarborough. This would result in the shortest path being found much quicker.

Empirically testing A* against Dijkstra's algorithm

You could empirically test the speed difference between A* and Dijkstra's algorithm by generating varying number of points in 2D space, with random edges going between the two and running the algorithm on

random starting and target nodes, comparing the execution times. It is important to note that for any node, its heuristic value must be a lower bound to the total weight of the path to the target node, otherwise A* could break and return a path that isn't the shortest.

Effects of an arbitrary heuristic function

If you generated an arbitrary heuristic function, A* could return the wrong result if the function isn't a lower bound of the total weight of the path to the target node. An arbitrary heuristic function could also push the algorithm to prioritize paths that aren't close to the shortest, which can take a lot longer then Dijktra's algorithm.

Applications where A* should be used instead of Dijkstra's algorithm

A* should be used in applications where there exists a heurstic function that is quick to compute, is always a lower bound to the weight of the shortest path from the inputted node to the target node, and the function decreases as you move to nodes closer to the target. Because straight line distances are always the lower bound to the shortest route, they are easy to calculate ($O(1)$ via pythagorean theorem if you have cartesian coordinates) and they decrease as you get closer to the target node. This is why A* is so good for shortest route calculations.

Part 3

Data and Graph Construction

We built a weighted graph from the London Tube data. Each station was treated as a node, and each connection between two stations was treated as an edge. Using `london_connections.csv`, we added an edge whenever two stations were connected in the dataset. The weight of each edge was based on the geographical distance between the two stations, which we computed using the latitude and longitude values in `london_stations.csv`. In our implementation, these connections were treated as bidirectional.

This gave us a real transportation network to work with instead of a random graph. Because the London Tube has both physical layout and line transfer structure, it was a good dataset for comparing Dijkstra's algorithm and A* in a meaningful way.

Heuristic Function

For each destination station d, we built a heuristic dictionary for A*. The heuristic value of a node was the straight line distance from that node to d. This matches the suggestion in the project description to use the physical "as the crow flies" distance as the heuristic.

This is a reasonable heuristic for the London Tube graph because every station has real coordinates. A station that is closer to the destination in physical distance is often a better direction to explore. Because of this, A* gets useful guidance toward the goal, while Dijkstra does not use this extra information.

Experimental Setup

We compared Dijkstra and A* on many source destination pairs in the London Tube graph. For each pair, we recorded the runtime of both algorithms, rebuilt the shortest path, measured the path length, and counted the minimum number of line transfers on that path. We grouped the journeys into three categories:

- **Same line**: 0 transfers,
- **One transfer**: 1 transfer,
- **Several transfers**: 2 or more transfers.

Before studying the performance results, we checked that the implementation was correct. Our test suite showed that the London data loaded correctly and that A* matched Dijkstra on sampled and additional source destination pairs. After generating the full `part3_results.csv` file, we also checked the entire output. This check found 0 distance mismatches between Dijkstra and A*, and 0 invalid rows with infinite distance or empty paths. Therefore, the results below are based on correct shortest path outputs.

Results

Figure 6 shows the median runtime of both algorithms grouped by shortest path length. In general, the median runtime increases as the path length increases. However, A* stays faster than Dijkstra across almost the whole range of path lengths. This shows that the geographical heuristic is still useful even for longer journeys.

Figure 5 shows the median runtime grouped by transfer category. Same line trips are the fastest for both algorithms, trips with one transfer are slower, and trips with several transfers are the slowest. In all three categories, A* has a lower median runtime than Dijkstra.

Figure 7 shows the speedup of A* over Dijkstra, where

$$\text{speedup} = \frac{\text{Dijkstra runtime}}{\text{A* runtime}}.$$

The median speedup is about 1.3× for same line trips, about 1.8× for trips with one transfer, and about 1.75× for trips with several transfers. This means that A* is faster in all three categories, and its advantage becomes larger when the trip is less direct.

We also looked at runtime grouped by the exact number of transfers. The same pattern appeared there too: as the number of transfers increased, the runtime of both algorithms usually increased, but A* still stayed faster than Dijkstra.

Discussion

Overall, our experiments show that A* was consistently faster than Dijkstra on the London Tube graph. The gap between the two algorithms was smallest for short same line trips, where both algorithms had less work to do. These were the cases where the two algorithms were the most similar. Even in these simpler cases, A* still had an advantage.

As trips became longer or needed more transfers, the difference between the two algorithms became larger. The main reason is the geographical heuristic used by A*. Since the graph has real coordinate data, the straight line distance to the destination gives a meaningful estimate of how far a node is from the goal. This helps A* move in the right direction, while Dijkstra explores the graph more evenly and may spend more time on nodes that do not help reach the destination quickly.

The transfer based results support this idea. Same line trips are usually simpler and have fewer possible route choices. Trips with one or several transfers are more complicated and create more possible intermediate routes. In these harder cases, the heuristic becomes more useful because it helps A* avoid wasting time on less relevant parts of the graph. This matches the speedup values in Figure 7, where A* shows a bigger advantage once the trip is no longer a direct same line journey.

The path length results show the same idea in another way. Shorter paths are usually easier because the algorithms can finish quickly. As the path length grows, the search space becomes larger and there are more choices to consider. Because of this, the median runtime of both algorithms increases. However, the increase is much larger for Dijkstra than for A*. This suggests that the heuristic becomes more useful as the trip becomes longer and less direct.

In our experiments, Dijkstra did not show strong evidence of being better than A* on typical journeys. The closest cases were short same line trips, where their runtimes were the most similar. For this dataset, A* appears to be the better practical choice when the goal is to find a path to one specific destination and a useful heuristic is available.

We included the line based part of the assignment by using the transfer count of the rebuilt shortest path. We treated 0 transfers as a same line trip, 1 transfer as a one transfer trip, and 2 or more transfers as a trip with several line changes. This helped us connect the algorithm results directly to the structure of the London Tube system instead of only describing the graph in a general way.

Limitations

One limitation of this experiment is that the edge weights were based on geographical distance instead of real travel time. Because of this, the graph does not include waiting time, different train speeds, station delays, or walking time during transfers. In the same way, the heuristic only uses straight line distance and does not directly include transfer cost. Even with these simplifications, the heuristic still gave A* a clear advantage on this dataset.

Conclusion

In conclusion, the London Tube experiments show that A* is usually the better choice when a useful heuristic is available. On this dataset, A* had lower median runtime than Dijkstra across different path lengths and across different transfer categories. The difference was smallest for simple same line trips and larger for trips with one or several transfers. These results support the idea that A* is especially useful on real transportation networks where a good lower bound heuristic can be computed easily.

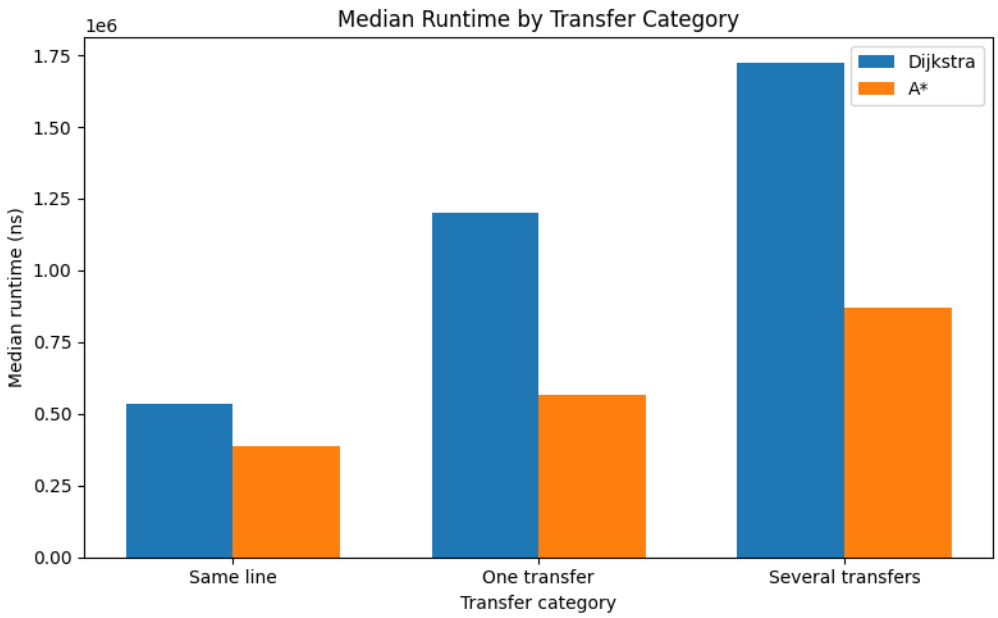

Figure 5: P3_Median runtime by transfer category. A* is faster in all three groups.

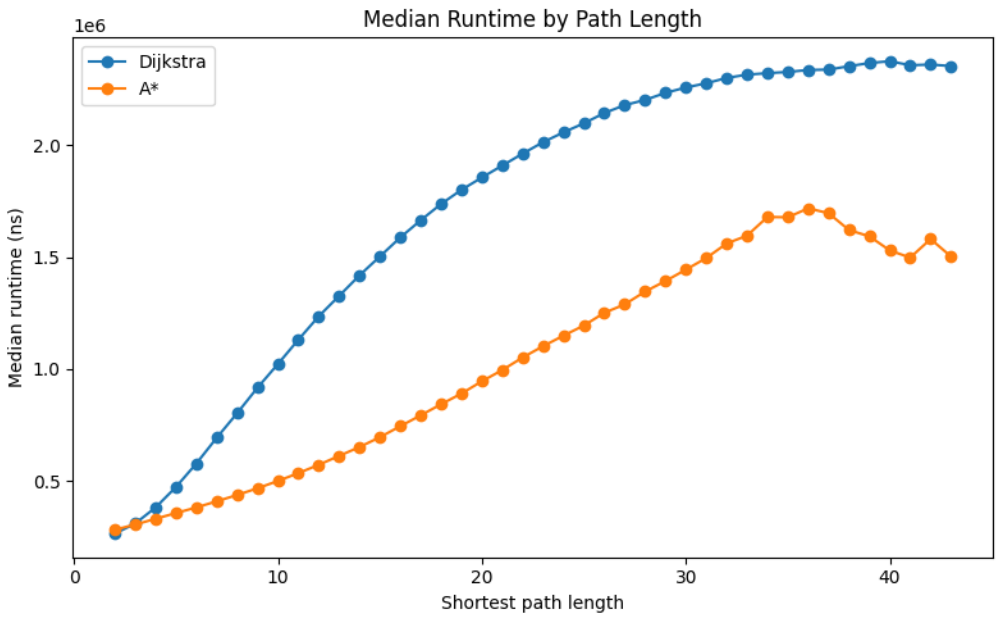

Figure 6: P3_Median runtime by path length. A* remains faster across almost all path lengths.

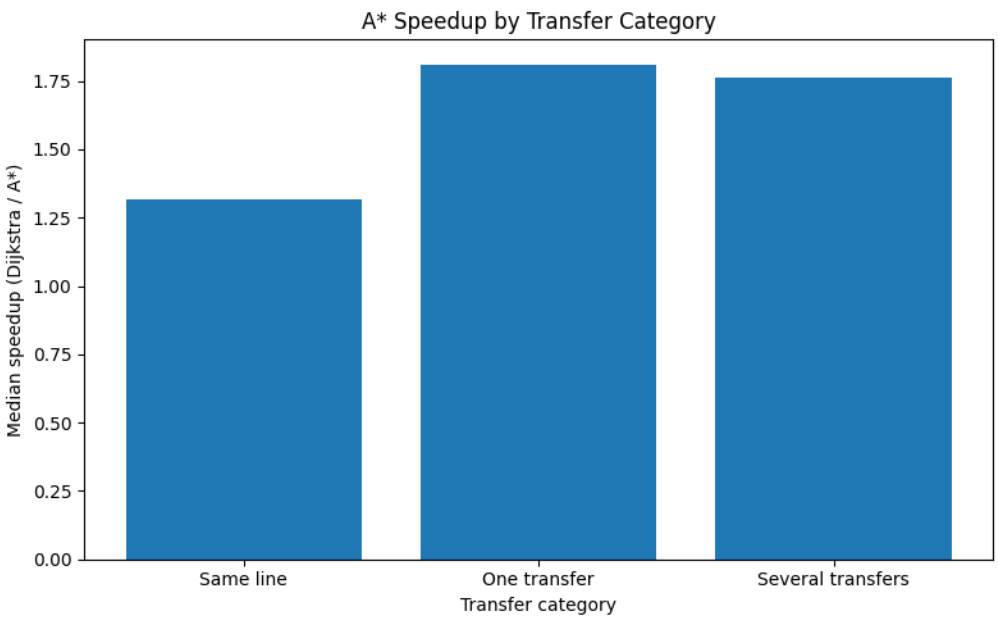

Figure 7: P3_A* speedup by transfer category.

Part 4

Discussion

Figure 2's design principles and patterns

In Figure 2, the design principles and patterns being used are:

- **Strategy and Template Design Pattern:** The SPAlgorithm is a template strategy for the strategies implementing the following algorithms: Dijkstra's, Bellman Ford, and A*
- **Adapter Design Pattern:** In the assignment instructions, A*'s strategy is supposed to be adapted from its implementation in **Part 2**. I also decided to adapt **Bellman_Ford** from its implementation in **Part 1**
- **Abstraction Design Princple:** The **ShortPathFinder** Class is abstracted from the implementation of the sorting algorithms and the graphs. This is because their innerworkings are encapsulated within their class and hidden from **ShortPathFinder**

Robustness of the future implementation's of different node representation

If in the future, we may need nodes to be represented as something other then integers, we can implement a class that is a decorator to **Graph**. All this new class would have is an instance of **Graph** or its children, and a dictionary that maps each node's in-graph integer representation to whatever you want the node represented as. You can then add a method to get the **Graph** instance to do shortest path calculations, and decorator methods to **Graph** (e.g. add nodes with one function call giving a name as a string, which adds an integer to the **Graph** instance and a mapping to the name in the dictionary).

This solution abstracts what nodes represent in a graph away from **Graph** and **SPAlgorithm**, keeping implementations of **Graph** and **SPAlgorithm** simple and avoiding the need to change their implementations.

Potential graphs that can be implemented

We can implement many other graphs such as:

- Directed Acyclic Graphs and Bipartite Graphs by adding a method that returns a boolean if it follows their rules and/or preventing changes from being made that break the rules.
- Undirected graph's by changing "add_edge" to add two directed edges going both ways.
- Multigraphs which can have multiple edges between two nodes. One way is if you want duplicate edges with different weights, the "w" method would need to return a list of floats. We would also need to update the "add_edge" method and how the weights are stored to support this.

Appendix

Additional Figures for Part 3

The following figures provide extra detail for the Part 3 analysis. They are included in the appendix because they are useful for completeness, but they are less central to the main conclusions than Figures 5, 6, and 7.

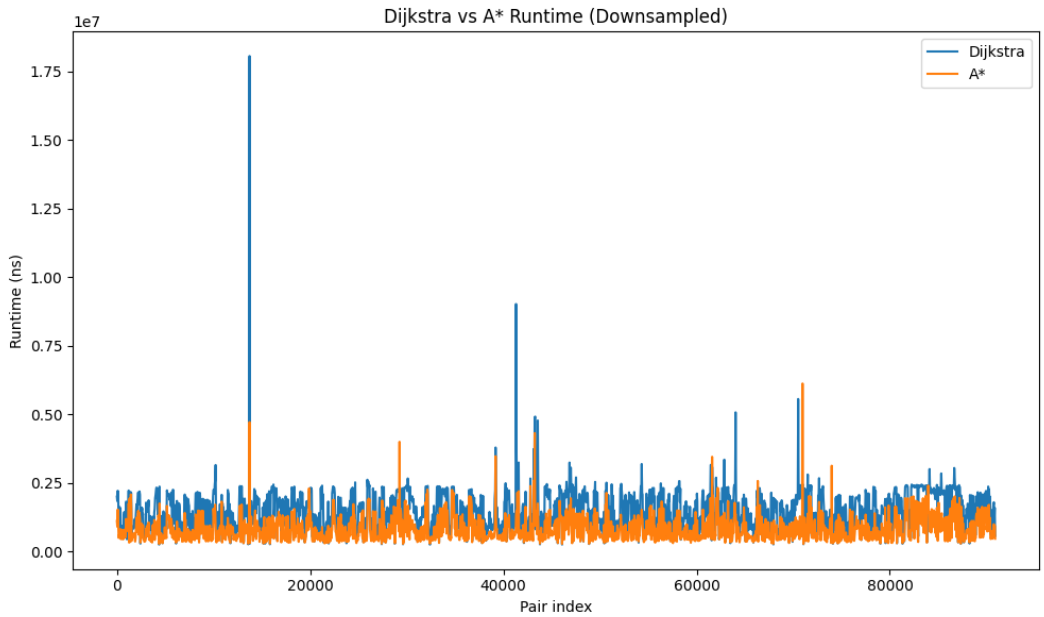

Figure 8: P3_Downsampled runtime comparison of Dijkstra and A*.

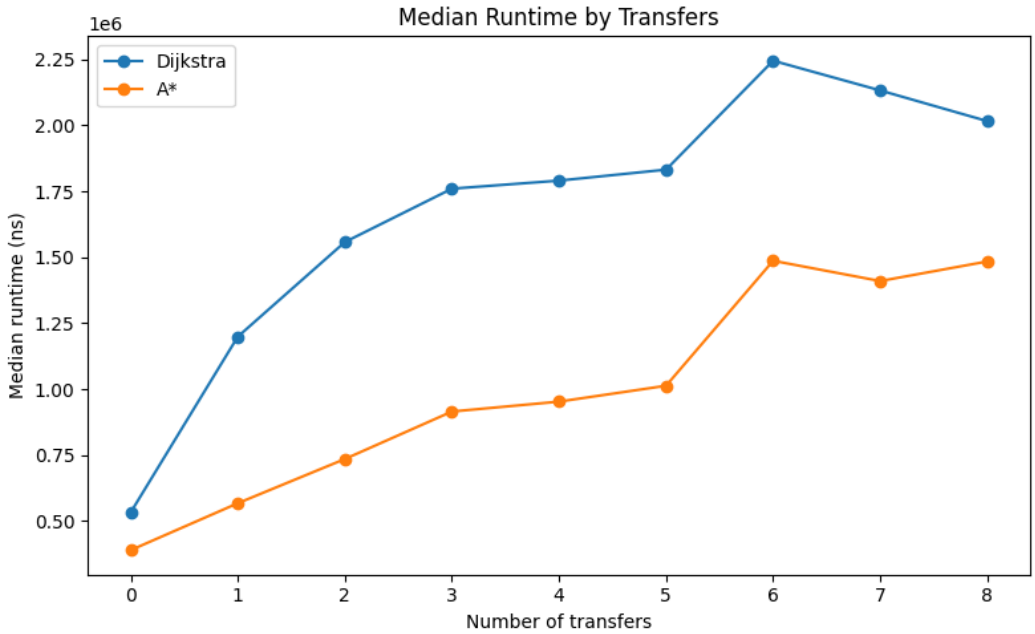

Figure 9: P3_Median runtime by exact number of transfers.

Code Navigation

The source code is organized systematically by part as follows:

Part 1: Traditional Algorithms and Approximations

- `final_project_part1.py`: Base graph class, Dijkstra, Bellman-Ford, and the mystery algorithm.
- `final_project_part1_approx.py`: Implements the k-relaxation limits for Dijkstra and Bellman-Ford.
- `final_project_part1_test1.py` to `test3.py`: Runs Experiment Suite 1 to test k values, densities, and sizes.
- `final_project_part1_mystery_test.py`: Runs Log-Log comparison to test runtimes of Bellman-Ford, Dijkstra, and Mystery algorithms.

Part 2: A* Algorithm Implementation

- `final_project_part2.py`: Implements the A* algorithm and the custom `AStarElement` priority queue element.
- `final_project_part2_test.py`: Validates A* correctness against Dijkstra using a sample graph.
- `final_project_part2_test_visualization.py`: Generates a visual plot of the sample test graph.

Part 3: London Tube Experiment

- `final_project_part3.py`: Loads London Tube data, builds the graph, and runs the all-pairs experiment.
- `final_project_part3_test.py`: Correctness tests for data loading and baseline shortest paths.
- `part_3_resultVerify.py`: Checks the generated CSV file for distance mismatches or invalid paths.
- `test_visualization.py`: Reads results and generates the performance figures used in this report.
- `london_stations.csv` & `london_connections.csv`: Datasets used to build the graph.
- `part3_results.csv`: Stores the raw output data from the main experiment.

Part 4: Code Refactoring and Design Patterns

- `final_project_part4.py`: Refactored OOP design, including `Graph`, `SPAlgorithm`, and `ShortPathFinder`.
- `final_project_part4_test.py`: Tests the refactored architecture to ensure all algorithms work correctly.